Optimising Tail Latency of Edge Applications

Student Details

Name	Roll Number
Shamik Sinha	2022468
Shrutya Chawla	2022487
Medha Kashyap	2022292
Shivankar Srijan Singh	2022479

BTP report submitted in partial fulfilment of the requirements
for the Degree of B.Tech. in Computer Science & Engineering

Date: April 22, 2026

BTP Track: Research

BTP Advisor

Arani Bhattacharya

Indraprastha Institute of Information Technology
Delhi

Student Declaration

We hereby declare that the work presented in the report entitled **"Optimising Tail Latency of Edge Applications"** submitted by us for the partial fulfilment of the requirements for the degree of *B.Tech. in Computer Science & Engineering* at Indraprastha Institute of Information Technology, Delhi, is an authentic record of our work carried out under the guidance of **Prof. Arani Bhattacharya**. Due acknowledgements have been given in the report for all material used. This work has not been submitted elsewhere for the award of any other degree.

Shamik Sinha

Shrutya Chawla

Medha Kashyap

Shivankar Srijan Singh

Place & Date: Delhi, April 22, 2026

Certificate

This is to certify that the above statement made by the candidates is correct to the best of my knowledge.

Dr. Arani Bhattacharya **Date:** April 22, 2026

Abstract

Edge computing reduces response latency for compute-intensive applications by moving processing closer to end users. However, the heterogeneous nature of edge infrastructure and the stochastic character of real-world workloads make it difficult to keep *tail latency*—the delays experienced by the slowest fraction of requests—consistently low. Even when average latency is acceptable, tail events caused by resource contention, queue build-up, or suboptimal routing can violate the hard deadlines of latency-sensitive services such as object detection, instance segmentation, and speech-to-text transcription.

This work presents **SafeTail**, a reinforcement-learning-based scheduling framework that addresses tail latency in heterogeneous edge environments. A central controller receives user service requests and dispatches them across a pool of heterogeneous edge servers. Rather than selecting a single best server, the controller may assign a request *redundantly* to multiple servers and accept the fastest response, a strategy that directly suppresses tail events. Scheduling decisions are made by a Deep Q-Network (DQN) whose input is processed by an integrated state encoder that maps variable-length, heterogeneous server-state vectors into a compact fixed-dimensional representation.

Queuing delays at both the controller and edge servers are modelled analytically as M/M/1 queues, enabling accurate waiting-time estimates without requiring explicit simulation. Execution times for individual tasks are predicted by Multi-Layer Perceptron (MLP) regressors trained on measurements collected from real heterogeneous hardware under concurrent workload conditions. The reward function combines a step-level resource-utilisation score with an episode-level degree of deadline satisfaction and a waiting-time penalty, steering the agent towards policies that simultaneously balance load and maximise the number of requests completed within their deadlines.

Experiments are conducted with 500,000 synthetic requests spanning three request types, each with type-specific soft and hard deadlines, served by five edge servers with different CPU, GPU, and memory configurations. Results demonstrate that the trained DQN agent achieves higher deadline satisfaction and lower tail latency compared to static baselines including minimum-load, minimum-propagation-delay, and random allocation strategies.

Keywords: Edge Computing, Tail Latency, Deep Reinforcement Learning, Task Scheduling, Heterogeneous Edge Devices, M/M/1 Queuing, Redundant Allocation, Deadline Satisfaction.

Acknowledgment

We would like to express our sincere gratitude to **Dr. Arani Bhattacharya** for his constant guidance, encouragement, and insightful feedback throughout the duration of this project. We are also deeply thankful to **Ms. Jyoti Shokhanda, Ph.D. student**, for her continuous support, technical discussions, and valuable inputs that greatly strengthened our work. Their mentorship enabled us to explore and learn several new technologies and concepts, ranging from reinforcement learning and queueing theory to system-level performance modeling and heterogeneous edge computing, which played a crucial role in successfully making progress in this project. We are truly grateful for the opportunity to work under their supervision and for the learning experience this project has provided .

Contents

Chapter 1

Introduction

The proliferation of latency-sensitive applications—augmented reality, real-time video analytics, autonomous systems, remote health monitoring, and intelligent IoT services—has exposed a fundamental limitation of centralised cloud computing: the round-trip latency to a distant data centre is simply too high for applications that require responses within tens of milliseconds. Edge computing addresses this by deploying computational resources at the network periphery, close to the devices that generate requests [9, 7]. While edge deployment reduces the propagation component of latency, it introduces a different set of challenges: edge clusters are heterogeneous (servers differ in CPU clock speed, number of cores, GPU capability, and available memory), workloads are bursty and unpredictable, and individual servers are resource-constrained compared with cloud data centres.

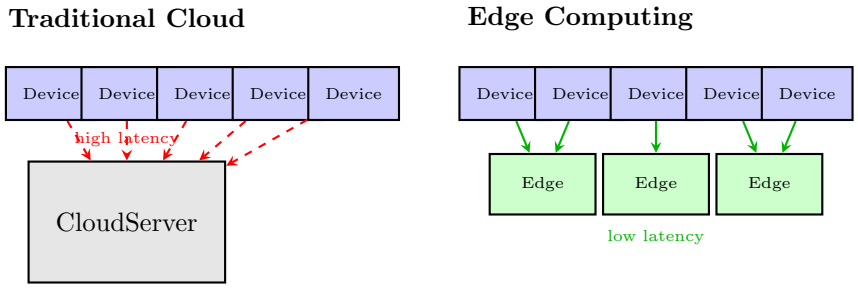


Figure 1.1: Traditional cloud versus edge computing.

The Tail Latency Problem

Scheduling requests across a heterogeneous edge cluster is a non-trivial optimisation problem. Even when the average response time is satisfactory, a small fraction of requests may experience disproportionately long delays—a phenomenon known as *tail latency* [2]. Sources include queue build-up at overloaded servers, resource contention from co-located workloads, and the inherent stochasticity of task execution times [6]. In safety-critical or interactive applications, a single missed deadline can degrade user experience or trigger a failure; therefore, optimising only the mean is insufficient.

Approach

This work proposes **SafeTail**, a controller-based scheduling framework in which a central controller intercepts all user requests and dispatches them to heterogeneous edge servers using a trained Deep Q-Network (DQN) agent. Two design choices directly target tail latency.

Redundant allocation. Instead of routing each request to a single server, the DQN may select a subset of servers and execute the request on all of them in parallel, accepting whichever response arrives first [11]. This converts an extreme-value problem (worst-case server) into a minimum-value problem, substantially reducing the probability of a deadline miss.

Learning-based scheduling. Rather than using static heuristics such as round-robin or minimum-load, the controller learns a scheduling policy through interaction with the environment. The policy accounts for the current resource utilisation of every server—CPU, GPU, and memory—as well as request-specific parameters.

Queuing delays are handled analytically through M/M/1 queue models, and task-execution times are predicted by lightweight MLP regressors trained on real hardware measurements. These two components allow the reward signal to incorporate accurate latency estimates without requiring a full discrete-event simulation.

Contributions

The specific contributions of this work are:

1. An end-to-end integrated Encoder+DQN architecture that processes variable-length, heterogeneous edge-state vectors and outputs a policy over all non-empty server subsets.
2. A two-level reward function combining a step-level resource-utilisation signal with an episode-level degree of deadline satisfaction and normalised waiting-time penalty.
3. A piecewise deadline-satisfaction function $P(T)$ that provides partial credit for requests completing between the soft deadline D_1 and hard deadline D_2.
4. Regression-based computation-latency models trained on real GPU/CPU hardware under concurrent workload scenarios, enabling realistic heterogeneous simulation.
5. Empirical comparison of the DQN policy against static baselines on three task types with different deadline budgets.

Report Structure

Chapter 2 surveys related work on edge scheduling, tail-latency mitigation, and deep reinforcement learning for resource management. Chapter 3 formally defines the system model and problem

statement. Chapter 4 describes the DRL architecture, reward design, and training procedure in detail. Chapter 5 presents experimental results. Chapter 6 concludes and discusses directions for future work.

Chapter 2

Related Work

2.1 Edge and Fog Computing

The concept of offloading computation from end devices to nearby infrastructure dates to Satyanarayanan et al.'s proposal of VM-based cloudlets [9], which argued that a one-hop wireless connection to a nearby server is preferable to a multi-hop path to a cloud data centre for latency-critical workloads. Mahmud et al. [7] survey the closely related fog computing paradigm, cataloguing deployment models, resource management challenges, and the tension between proximity (low latency) and capacity (cloud scale). Xu et al. [13] provide a broader view of edge intelligence architectures, highlighting heterogeneity as the defining characteristic that distinguishes edge clusters from homogeneous cloud back-ends.

The present work targets this heterogeneity explicitly: the five simulated edge servers differ in CPU frequency, core count, GPU memory, and current utilisation, reflecting the diversity of real-world deployments.

2.2 Tail Latency in Distributed Systems

Dean and Barroso [2] showed that, in large-scale services, even a small probability of a slow component can significantly degrade end-to-end latency at the tail (99th percentile), because a request that fans out to many components is only as fast as its slowest component. Li et al. [6] identify hardware, OS, and application-level sources of tail latency, including CPU frequency scaling, interrupt coalescing, and garbage collection pauses. Their analysis motivates the need for scheduling strategies that are aware of server state rather than treating servers as identical.

2.3 Redundancy-Based Latency Mitigation

A practical approach to tail latency is redundant execution: issue the same request to multiple servers and use the first response. Vulimiri et al. [11] analyse this strategy theoretically, showing

that even modest redundancy (issuing to two servers instead of one) can substantially cut tail latency when service times are heavy-tailed. Ananthanarayanan et al. [1] implement a related idea in Hadoop with speculative execution—re-launching slow tasks on idle machines.

SafeTail adopts subset-based redundant allocation but makes it adaptive: the DQN agent learns which server subsets to activate based on current resource states, rather than always replicating to a fixed number of servers.

2.4 Deep Reinforcement Learning for Resource Management

Mao et al. [8] demonstrate that a neural network policy trained with REINFORCE can significantly outperform heuristics for job-shop scheduling in data centres, motivating the application of DRL to edge resource management. Wei et al. [12] apply DQN to mobile edge computing (MEC) task offloading, jointly optimising energy and delay. He et al. [4] extend this to edge-cloud collaborative settings, where the agent must choose between local edge processing and cloud offloading under varying network conditions.

A recurring challenge in these works is state representation: server states are high-dimensional and heterogeneous, making direct concatenation into a fixed-size vector impractical. SafeTail addresses this with a learned encoder that maps variable-length state sequences to a compact fixed-dimensional embedding before passing them to the DQN.

2.5 Queuing Models for Edge Latency

Analytical queuing models are widely used to estimate waiting times in edge systems without full simulation. The M/M/1 queue—Poisson arrivals, exponential service times, single server—provides a tractable closed-form expression for mean waiting time [5]. Harchol-Balter [3] provides a comprehensive treatment of performance modelling techniques applicable to computer systems. SafeTail uses M/M/1 queue models at both the controller and each edge server to compute expected waiting times as part of the latency estimate fed into the reward function.

2.6 Computation Latency Prediction

Predicting task execution time is important for accurate reward computation. Zhang et al. [14] train neural networks to predict DNN inference latency from model and hardware characteristics. SafeTail uses task-specific MLP regressors trained on empirical measurements of three workload types (object detection with YOLO, instance segmentation with Mask R-CNN, and speech-to-text with Whisper) under varying levels of concurrent load on real GPU/CPU hardware. This captures the resource-contention effects that are absent from analytical models.

2.7 Controller-Based Scheduling and Heterogeneous Edge Resource Allocation

Recent work has also explored controller-centric scheduling architectures for heterogeneous edge systems, where a central controller dynamically routes incoming jobs to suitable edge devices based on their instantaneous state. In this model, the controller maintains both static resource descriptors (e.g., CPU speed, memory capacity, GPU memory, number of cores) and dynamic utilisation metrics (CPU load, GPU load, memory usage, active users, bandwidth), and uses these observations to make task-placement decisions. To address the varying dimensionality of heterogeneous server states, an encoder is proposed to map each device state into a fixed-length representation before passing it to a reinforcement learning scheduler. The scheduler can select multiple edge devices simultaneously for redundant execution, while queueing delay at both the controller and servers is modelled using M/M/1 queues. Experimental evaluation across object detection, instance segmentation, and speech workloads demonstrates the importance of modelling resource contention, heterogeneous hardware, and controller bottlenecks jointly when optimising latency-sensitive services. This line of work strongly aligns with SafeTail's assumptions of a central scheduler, heterogeneous server states, learned state embeddings, and redundancy-aware placement decisions, while SafeTail further extends the objective toward explicit tail-latency minimisation rather than only average delay or deadline satisfaction [10].

Chapter 3

System Model and Problem Statement

3.1 System Components

The system consists of three types of entities:

- A set of n users $\mathcal{U} = \{u_1, u_2, \ldots, u_n\}$ that generate service requests.
- A **central controller** that intercepts all requests and makes scheduling decisions.
- A set of m **heterogeneous edge servers** $\mathcal{E} = \{e_1, e_2, \ldots, e_m\}$ that execute the tasks.

Users communicate with the controller over a wireless network. The controller communicates with edge servers over a faster, more reliable wired network. Edge servers notify the controller upon job completion so that the controller maintains an accurate picture of server load.

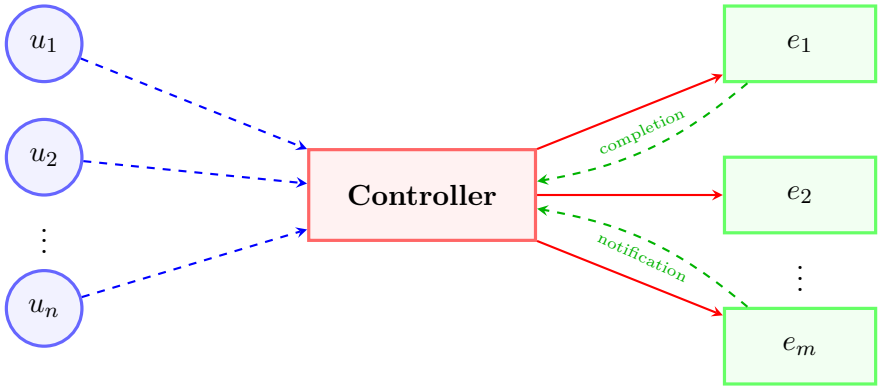


Figure 3.1: System architecture: users, controller, and heterogeneous edge servers.

3.2 Edge Server State

Each server e_i is characterised by a state vector $\mathcal{S}_i^{(t)}$ composed of both static and dynamic attributes.

Static attributes (hardware constants):

- CPU clock speed and number of cores
- Total RAM capacity
- GPU clock speed and total GPU memory

Dynamic attributes (time-varying):

- Transmission Delay
- Propagation Delay
- Current load L_i^t (number of active concurrent requests) and maximum capacity $L_{\max,i}$
- CPU, GPU, and memory utilisation percentages
- Uplink and downlink bandwidths $\lambda_i^{u(t)}, \lambda_i^{d(t)}$

Because servers have different hardware configurations, these vectors have different dimensions across servers. An encoder network (Section 4.3) is used to project them into a common fixed-dimensional space before scheduling decisions are made.

3.3 Request Model

Each service request s_j is characterised by:

- A **type** $\tau_j \in \{\texttt{s}, \texttt{d}, \texttt{p}\}$, Object detection using YOLOv5 (d), Instance segmentation of images(s), and Removal of noise from audio(p).
- Input and output data sizes that determine transmission delay.
- Computational characteristics (FLOPs, input dimensions) that influence execution time.
- A deadline pair $(D_1^{\tau_j}, D_2^{\tau_j})$, where D_1 is a soft deadline and $D_2 > D_1$ is a hard deadline (Table 3.1).

Type	D_1 (ms)	D_2 (ms)
Speech (`s`)	100	400
Detection/Predict (`d`/`p`)	30	200

Table 3.1: Deadline parameters used in experiments.

3.4 Latency Model

The total end-to-end latency experienced by request s_j is:

$$L_j = T_{\text{transmission}} + T_{\text{ctrl-queue}} + T_{\text{propagation}}$$

- $T_{\text{transmission}}$: transmission delay from user to controller plus from controller to user
- $T_{\text{ctrl-queue}}$: waiting time in the controller's queue before scheduling.
- $T_{\text{propagation}}$: propagation delay from controller to the assigned edge server(s) and from server back to user.
- T_{exec}: task execution time on the assigned server (predicted by MLP regressor).

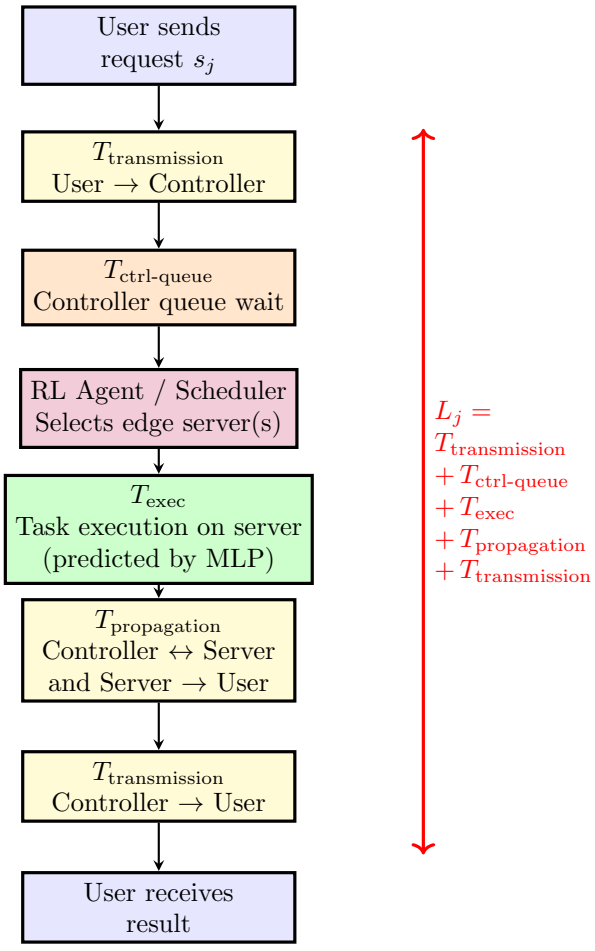


Figure 3.2: Updated end-to-end latency model for request s_j.

3.5 Queuing Model

Both the controller and each edge server are modelled as **M/M/1 queues**: requests arrive according to a Poisson process with rate λ and service times are exponentially distributed with rate μ. The

expected waiting time in such a queue is [5]:

$$W = \frac{\lambda}{\mu(\mu - \lambda)}, \qquad \mu > \lambda$$

When the DQN selects a subset $\mathcal{A} \subseteq \mathcal{E}$ of servers for request s_j, the effective waiting time is taken as the minimum over the selected servers:

$$W_j = \min_{e_i \in \mathcal{A}} W_i$$

This directly models the benefit of redundant allocation: the request completes as soon as the fastest server in the subset finishes.

3.6 Step and Episode Definitions

Step. A step corresponds to the scheduling of one chunk of k requests. The controller receives the chunk, the DQN selects a server subset for each request, execution begins, and per-step rewards are computed.

Episode. An episode consists of a fixed number of consecutive steps. At the end of an episode, the accumulated step rewards and deadline-satisfaction statistics are combined into an episodic reward, experiences are stored in the replay buffer, and the DQN is updated.

3.7 Deadline Satisfaction

For a request with completion time T and deadlines (D_1, D_2), the **degree of satisfaction** $P(T)$ is:

$$P(T) = \begin{cases} 1 & T \leq D_1 \\ 1 - \dfrac{T - D_1}{D_2 - D_1} & D_1 < T \leq D_2 \\ 0 & T > D_2 \end{cases}$$

$P(T) = 1$ denotes full satisfaction (response within the soft deadline); $P(T) = 0$ denotes complete dissatisfaction (response after the hard deadline); the intermediate region provides partial credit that decreases linearly with latency. The episode-level **aggregate satisfaction** is:

$$\omega = \frac{1}{N} \sum_{j=1}^{N} P(T_j)$$

where N is the expected number of requests per episode.

3.8 Problem Statement

Given m heterogeneous edge servers with time-varying resource states and n users generating requests with type-specific deadline constraints, the goal is to find a scheduling policy $\pi : \mathcal{S} \to 2^{\mathcal{E}} \setminus \emptyset$ that maps the current system state to a non-empty subset of edge servers, such that:

1. **Tail latency is minimised** by redundant allocation to multiple servers.
2. **Deadline satisfaction ω is maximised** across all request types.
3. **Load is balanced** by avoiding repeated assignment to already-congested servers.
4. **Long-term cumulative reward is maximised**, accounting for the discounted sum of per-step quality metrics.

The exponential action space ($|\mathcal{A}| = 2^m - 1$ non-empty subsets) and the continuous, partially observable server state make exact optimisation intractable, motivating a deep reinforcement learning approach.

Chapter 4

Methodology

4.1 Overview

SafeTail frames the scheduling problem as a Markov Decision Process (MDP) and trains a Deep Q-Network (DQN) to approximate the optimal action-value function. The agent observes the current resource state of all edge servers, selects a subset of servers to handle each request, receives a reward signal that reflects both resource utilisation and deadline satisfaction, and updates its policy through experience replay. Figure 4.1 illustrates the high-level control loop.

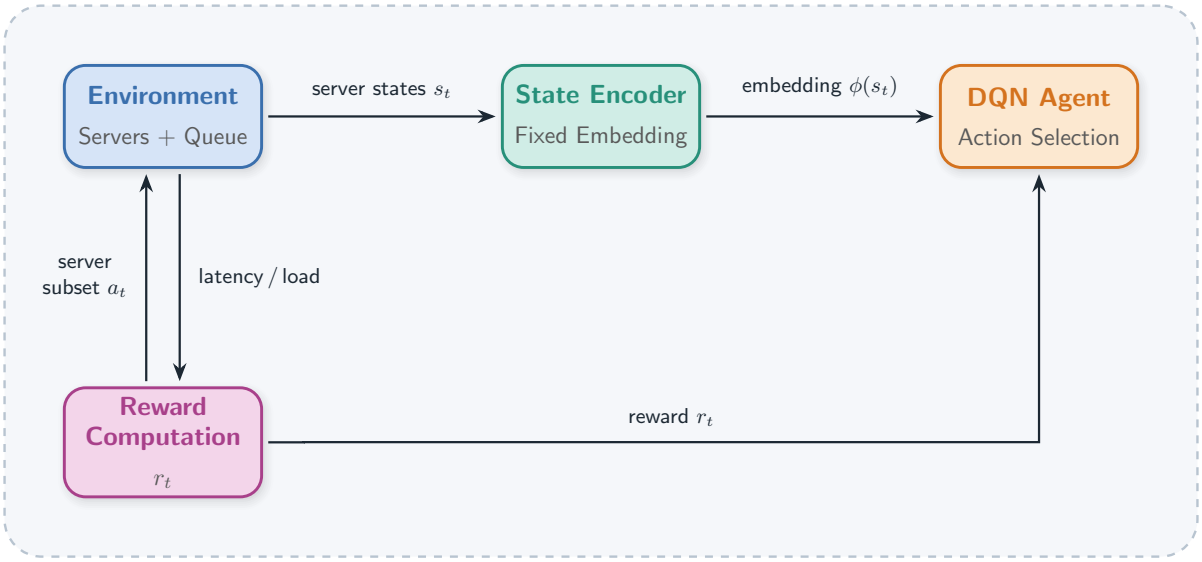


Figure 4.1: SafeTail control loop.

4.2 MDP Formulation

State. At each step t, the state s_t encodes the current load on each of the β edge servers as a fraction of maximum capacity, together with a global utilisation indicator. This yields a $(\beta + 1)$-dimensional feature vector per step. Across an episode, these per-step vectors are accumulated and

treated as a variable-length sequence, allowing the encoder to capture temporal patterns in server load.

Action. An action $a \in \{1, \ldots, 2^\beta - 1\}$ identifies one of the $2^\beta - 1$ non-empty subsets of β edge servers. For $\beta = 5$, this gives $|\mathcal{A}| = 31$ discrete actions.

Reward. The reward signal operates at two levels, described in detail in Section 4.5.

Transition. After the selected servers process their assigned requests, server loads update, new requests arrive, and the state transitions to s_{t+1}.

4.3 Integrated Encoder + DQN Architecture

Edge server states are heterogeneous in dimension: servers differ in the number of attributes they expose. Concatenating raw state vectors directly would produce inputs of varying length that a standard fixed-input DQN cannot handle. To address this, the Q-network integrates a learned encoder that maps any variable-length state sequence to a fixed 24-dimensional embedding before the DQN layers process it.

The complete architecture is implemented as a single Keras model trained end-to-end, totalling 9,799 trainable parameters. Table 4.1 summarises each layer.

Block	Layer	Output shape
Encoder	Dense(64, ReLU)	$(B, T, 64)$
	Dropout	$(B, T, 64)$
	Dense(32, ReLU)	$(B, T, 32)$
	Dropout	$(B, T, 32)$
	GlobalAveragePooling1D	$(B, 32)$
	Dense(24, ReLU)	$(B, 24)$
DQN	Dense(48) + BatchNorm + Dropout	$(B, 48)$
	Dense(64) + BatchNorm + Dropout	$(B, 64)$
	Dense(31, linear)	$(B, 31)$

Table 4.1: Integrated Encoder + DQN layer summary. B = batch size, T = variable sequence length.

4.3.1 Encoder

The encoder treats the variable-length state sequence as a 1-D sequence of scalar features. Two Dense layers with ReLU activation and Dropout regularisation project each element to a 32-dimensional space. A `GlobalAveragePooling1D` layer then averages across the sequence dimension, producing a single 32-dimensional vector regardless of the original sequence length. A final Dense(24) layer yields the encoded state $\mathbf{z} \in \mathbb{R}^{24}$.

4.3.2 DQN

The encoded state $\mathbf{z}$ is passed through two fully-connected layers (sizes 48 and 64) with Batch Normalisation and Dropout, followed by a linear output layer with 31 units—one per non-empty server subset. Batch Normalisation improves training stability across the wide range of reward magnitudes that arise during early exploration.

4.4 Epsilon-Greedy Exploration and Training

The agent follows an ε-greedy policy: with probability ε it selects a random server subset (exploration), and with probability $1 - \varepsilon$ it selects the action with the highest predicted Q-value (exploitation). ε decays from 1.0 towards $\varepsilon_{\min} = 0.1$ during training:

$$\varepsilon_{t+1} = \max(\varepsilon_{\min},\ \varepsilon_t \cdot (1 - \gamma_\varepsilon))$$

where γ_ε is the decay rate. Once $\varepsilon_{\min}$ is reached, the agent continues training in a post-exploration phase that accumulates an additional $N_{\text{post}} = 8{,}000$ steps before training concludes.

Experiences (s_t, a_t, r_t, s_{t+1}) are stored in a replay buffer. At the end of each episode, a mini-batch of 128 transitions is sampled uniformly and used to update the network weights via the Bellman equation:

$$Q(s,a) \ \leftarrow\ Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

with learning rate $\alpha = 10^{-6}$ and discount factor $\gamma = 0.9$. The learning rate decays multiplicatively each episode ($\text{lr} \leftarrow \text{lr} \times 0.999$, floored at 10^{-5}) to improve late-stage convergence.

Table 4.2 summarises the key hyperparameters.

Hyperparameter	**Value**
Number of edge servers β	5
Action space size $\lvert\mathcal{A}\rvert$	31
Initial ε	1.0
Minimum ε	0.1
Discount factor γ	0.9
Learning rate (initial)	10^{-6}
LR decay rate	0.999
Batch size	128
Total training episodes	25,000
Total requests	500,000

Table 4.2: Training hyperparameters.

4.5 Reward Function

The reward signal operates at two levels: a **step reward** computed per scheduling decision, and an **episodic reward** computed at the end of each episode.

4.5.1 Step Reward

For each request scheduled to server subset $\mathcal{A}$, the step reward measures how much *free* resource capacity is available across the selected servers. For each server $e_i \in \mathcal{A}$, four resource-utilisation fractions are computed:

- $c_m^{(i)}$ = RAM utilisation = used RAM / total RAM
- $c_u^{(i)}$ = CPU utilisation = (mean CPU core %) / 100
- $g_m^{(i)}$ = GPU memory utilisation = peak GPU memory used / total GPU memory
- $g_u^{(i)}$ = GPU utilisation percentage / 100

The per-server step reward is the product of available fractions:

$$R_{\text{step}}^{(i)} = \log\Big(1 + (1 - c_m^{(i)})\,(1 - c_u^{(i)})\,(1 - g_m^{(i)})\,(1 - g_u^{(i)})\Big)$$

This quantity equals 1 when the server is completely free and approaches 0 as any resource becomes saturated. The step rewards are accumulated across all steps in the episode and discounted for use in the episodic reward.

4.5.2 Episodic Reward

At the end of an episode, the episodic reward combines three components:

$$R_{\text{episode}} = \sum_{k=0}^{K-1} \gamma^k R_{\text{step},k} \;+\; \omega \;-\; \frac{\bar{W}}{W_{\text{norm}}} \tag{4.1}$$

where:

- K is the number of steps in the episode.
- $\omega = \frac{1}{N}\sum_j P(T_j)$ is the mean degree of deadline satisfaction over all N requests, with $P(T)$ as defined in Chapter 3.
- $\bar{W}$ is the mean queue waiting time (in seconds) across all requests.

- W_{norm} is a type-weighted normalisation factor based on the soft deadlines of completed requests, preventing the waiting-time penalty from dominating.

The raw value is clipped to $[-10, 10]$ to prevent Q-target divergence during early training.

The three terms serve distinct learning objectives: the discounted step-reward sum encourages resource efficiency at every step; ω provides a strong end-of-episode signal for deadline satisfaction; and the normalised waiting-time penalty discourages routing requests to congested queues even when execution eventually succeeds.

4.6 Computation Latency Prediction

Task execution times depend on the specific request type, the hardware of the target server, and the number of concurrently executing tasks. These dependencies are captured by task-specific MLP regressors trained offline on empirical measurements.

Task types and models:

- `s` — Instance segmentation of images
- `d` — Object detection (YOLOv5)
- `p` — Removal of noise from audio

For each of the five edge servers, separate regressors are trained per task type using measurements collected by running each workload at concurrency levels 1–4 on real hardware. Features include task-specific characteristics (input size, model complexity) and concurrent load indicators (CPU utilisation, GPU utilisation, number of active jobs).

During inference, the controller queries the appropriate regressor to predict T_{exec} for a request-server pair before committing to a scheduling decision, providing an accurate latency estimate for reward computation.

4.7 Baseline Comparison Methods

Results are compared against the following static scheduling policies:

- **minload**: Route each request to the server with the lowest current load.
- **minprop**: Route each request to the server with the shortest propagation delay.
- **rand**: Assign each request to a uniformly random server.

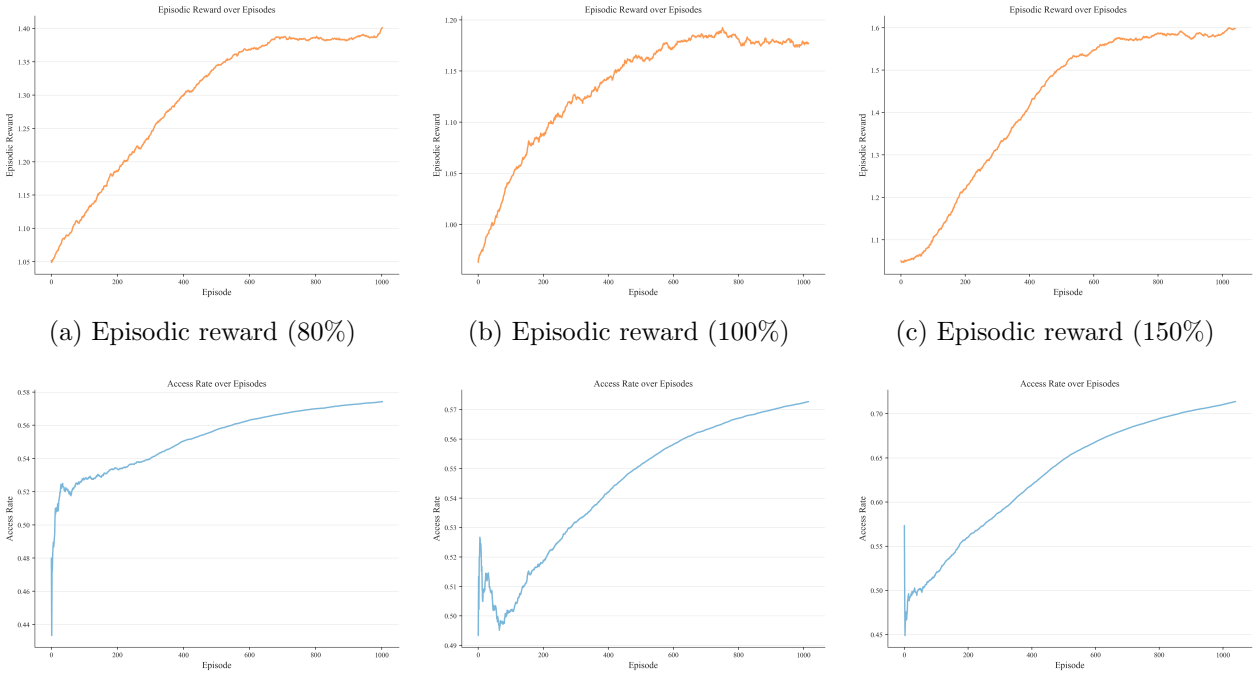
Each baseline is evaluated independently under the same workload trace to enable a controlled comparison.

Chapter 5

Evaluation and Results

5.1 Training Convergence

Figure 5.1 shows the training curves for SafeTail across three deadline configurations (80%, 100%, and 150% of the original deadlines). Each plot reports the exponentially-weighted moving average of the episodic reward and the cache access rate over the course of training.



(a) Episodic reward (80%) (b) Episodic reward (100%) (c) Episodic reward (150%)

(d) Access rate (80%) (e) Access rate (100%) (f) Access rate (150%)

Figure 5.1: Training convergence of SafeTail under three deadline configurations (80%, 100%, and 150% of the original deadlines). The top row shows episodic reward and the bottom row shows cache access rate, both smoothed with an exponential moving average for clarity.

Each request is associated with a soft deadline D_1 and a hard deadline D_2, as defined earlier. In our experiments, we scale these deadlines by 80%, 100%, and 150% to simulate stricter and more relaxed timing constraints, respectively.

5.2 Tail Latency Comparison with Baselines

We compare SafeTail against three baseline routing strategies: MinLoad, MinProp, and Random. For each strategy, we consider three variants based on the number of candidate servers selected from a total pool of five servers: **Variant-1, Variant-2, and Variant-3 correspond to selecting the top 1, 2, and 3 servers, respectively**. Specifically, MinLoad selects the least loaded servers, MinProp selects servers with the lowest propagation delay, and Random selects a subset of 1, 2, or 3 servers uniformly at random from the five available servers. The selected subset is then passed through the same downstream decision pipeline as SafeTail for final request assignment.

Figure 5.2 compares the end-to-end latency at the 50th, 90th, 95th, and 99th percentiles for SafeTail against these baselines under varying deadline configurations.

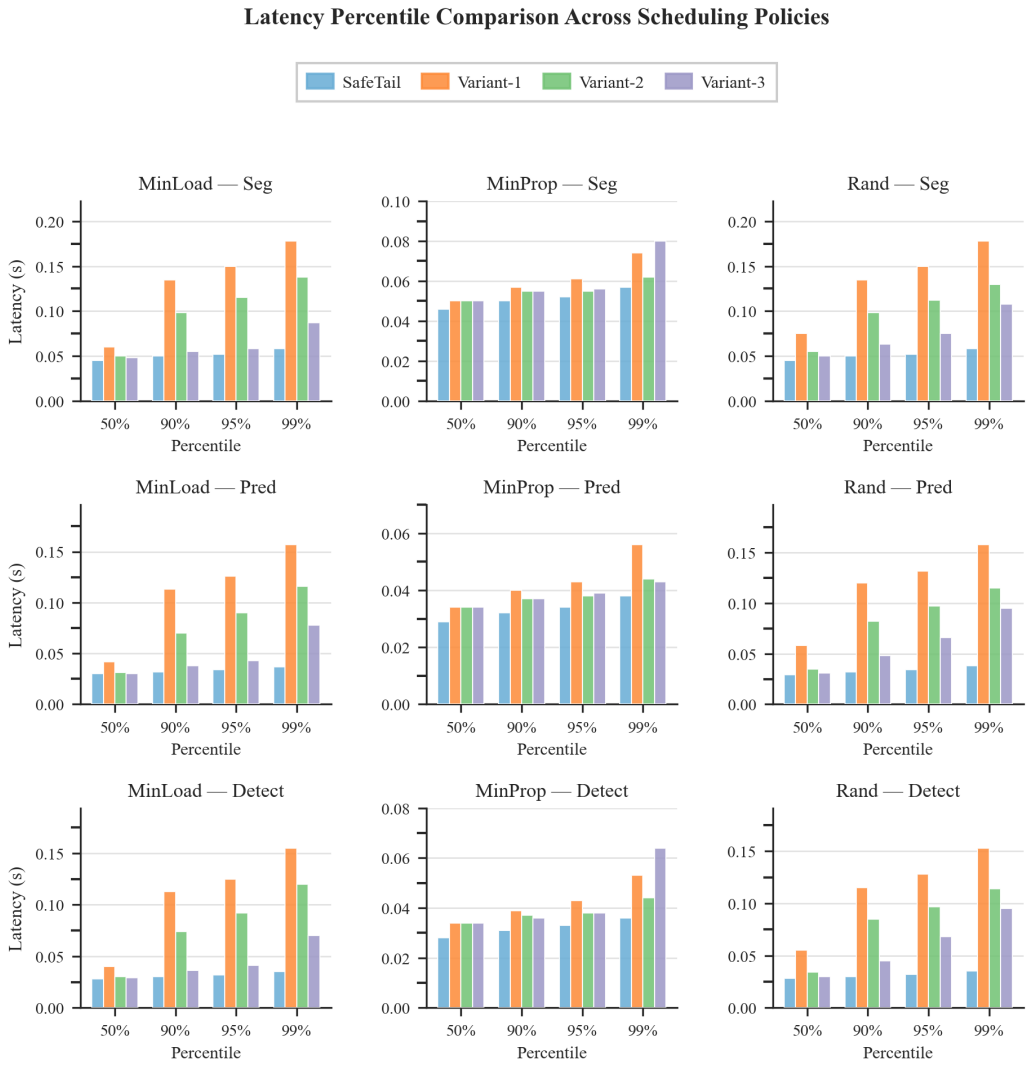
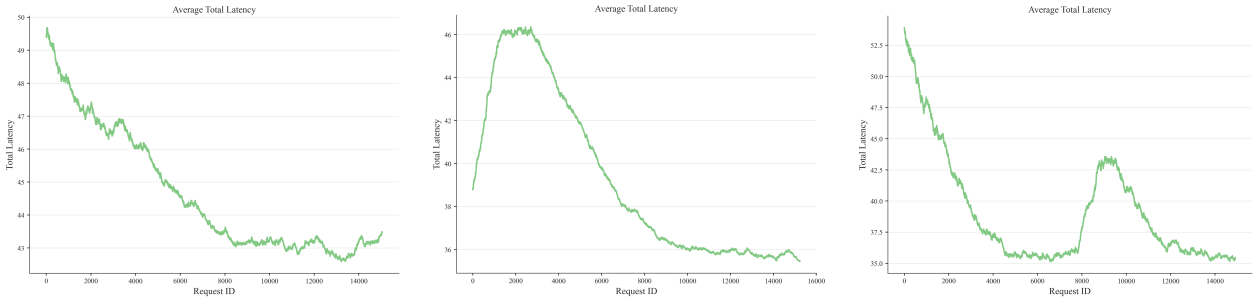


Figure 5.2: Latency percentile comparison across scheduling policies under varying deadline configurations. SafeTail consistently achieves lower tail latency compared to all baseline variants.

5.3 Total Latency over Time

Figure 5.3 shows the smoothed average total latency per request as a function of request ID across different deadline configurations.



(a) 80% deadline (b) 100% deadline (c) 150% deadline

Figure 5.3: Average total latency per request (smoothed) under three deadline configurations (80%, 100%, and 150% of the original deadlines).

Further interpretation and discussion of these results are presented in the Conclusion section.

Chapter 6

Conclusion

6.1 Summary

This work presented SafeTail, a deep reinforcement learning framework for tail-latency-aware task scheduling in heterogeneous edge computing environments. The core contribution is an integrated Encoder+DQN architecture that learns a scheduling policy over all non-empty subsets of edge servers without requiring a fixed-size, homogeneous state representation. The state encoder maps variable-length server-state sequences to a compact 24-dimensional embedding via Dense layers and global average pooling, enabling the DQN to generalise across servers with different hardware configurations.

The reward function combines three objectives: a step-level resource-utilisation score that encourages avoiding saturated servers; an episode-level deadline-satisfaction term ω derived from a piecewise function $P(T)$ that provides partial credit for latency values between the soft and hard deadlines; and a normalised waiting-time penalty that discourages routing to congested queues. Taken together, the reward steers the agent towards policies that are simultaneously load-aware and deadline-aware.

Execution-time uncertainty, one of the main drivers of tail latency, is addressed through per-server, per-task-type MLP regressors trained on real hardware measurements under varying concurrency conditions. Queuing delays are handled analytically with M/M/1 queue models, providing accurate waiting-time estimates at negligible run-time cost. Redundant allocation to multiple servers, with the controller accepting the first response, directly converts worst-case execution-time behaviour into a minimum-of-many problem, which is the most effective known countermeasure against tail latency [11].

6.2 Results Interpretation

The episodic reward increases over training and becomes stable for all deadline settings, which means the model learns to choose servers that meet deadlines and avoid heavy load. The reward

is highest for the 150% deadlines, which means more requests are completed on time and with less waiting. The access rate varies with deadline settings, being higher for relaxed deadlines (150%) and lower for stricter ones. This indicates that the model adapts the number of candidate servers per request, using greater redundancy when there is flexibility and being more selective under tighter constraints. The total latency decreases during training and then stabilises, which means requests are completed faster with lower waiting time in queues. The 150% setting gives the lowest latency, while the 80% setting remains higher because the deadlines are stricter. In the percentile comparison, SafeTail has lower latency than MinLoad, MinProp, and Random at all percentiles, with larger gaps at the 95th and 99th percentiles, which means fewer very slow requests.

6.3 Link to the Project Codebase

All of the code related to the above study can be found at this github link: SafeTail-2.0

6.4 Limitations and Future Work

Single controller. The centralised controller is a single point of failure and a potential bottleneck at high request rates. Distributing the controller or using a hierarchical design would improve resilience and scalability.

Transfer and generalisation. The policy is trained on a fixed set of five servers. Investigating transfer learning to new server configurations, without full re-training, is an important step towards practical deployment.

Bibliography

[1] Ganesh Ananthanarayanan et al. "Effective straggler mitigation: Attack of the clones". In: *Proceedings of the 10th USENIX Conference on Networked Systems Design and Implementation*. 2013, pp. 185–198.

[2] Jeffrey Dean and Luiz André Barroso. "The tail at scale". In: *Communications of the ACM* 56.2 (2013), pp. 74–80.

[3] Mor Harchol-Balter. *Performance modeling and design of computer systems: queueing theory in action*. Cambridge University Press, 2013.

[4] Yuxin He et al. "Deep reinforcement learning based edge-cloud collaborative computing for mobile users". In: *IEEE Transactions on Mobile Computing* 20.11 (2021), pp. 3229–3242.

[5] Leonard Kleinrock. *Queueing systems, volume 1: Theory*. John Wiley & Sons, 1975.

[6] Jialin Li et al. "Tales of the tail: Hardware, OS, and application-level sources of tail latency". In: *Proceedings of the ACM Symposium on Cloud Computing*. 2014, pp. 1–14.

[7] Redowan Mahmud, Ramamohanarao Kotagiri, and Rajkumar Buyya. "Fog computing: A taxonomy, survey and future directions". In: *Internet of Everything* (2018), pp. 103–130.

[8] Hongzi Mao et al. "Resource management with deep reinforcement learning". In: *Proceedings of the 15th ACM Workshop on Hot Topics in Networks*. 2016, pp. 50–56.

[9] Mahadev Satyanarayanan et al. "The case for VM-based cloudlets in mobile computing". In: *IEEE Pervasive Computing* 8.4 (2009), pp. 14–23.

[10] Jyoti Shokhanda et al. "SafeTail: Tail Latency Optimization in Edge Service Scheduling via Redundancy Management". In: *IEEE Transactions on Network and Service Management* (2025). DOI: 10.1109/TNSM.2025.3587752.

[11] Ashish Vulimiri et al. "Low latency via redundancy". In: *Proceedings of the 9th ACM Conference on Emerging Networking Experiments and Technologies*. 2013, pp. 283–294.

[12] Yutao Wei et al. "Deep reinforcement learning for task offloading in mobile edge computing systems". In: *IEEE Transactions on Mobile Computing* 20.2 (2019), pp. 748–763.

[13] Dianlei Xu et al. "Edge intelligence: Architectures, challenges, and applications". In: *Proceedings of the IEEE*. Vol. 107. 8. 2019, pp. 1493–1520.

[14] Li Zhang et al. "DNN execution time prediction using deep learning". In: *Proceedings of the International Conference on Machine Learning*. 2019, pp. 7345–7355.